

PA1 Technical Write-up: 3D Shapes and Graphics Pipeline in WebGL 1.0

Course: AR/VR/XR Applications (6B04103 AI Business, Year 3)

Author: Saule Amangeldyyeva | Student ID: 241487

1. Variant & Pipeline Configuration

- **Variant Parameter Mapping:**
 - **Second-to-last digit (8):** $8 \bmod 4 = 0 \rightarrow$ Screen-space Depth Offset Vector (ox, oy) = (+0.15, +0.15). Under active depth testing, visible cube faces comprise: Front, Top, and Right.
 - **Final digit (7):** Assigned Regular Polyhedron: Octahedron (8 triangular faces, 24 distinct vertices).
- **Canonical Model Domain:** All shape geometries are normalized within local bounds z in [-0.5, +0.5] and clipped canonically within clip space $[-1.0, 1.0]^3$.

2. Experimental Evaluations (E1 - E6)

E1 - Depth Testing and Draw Call Sequence

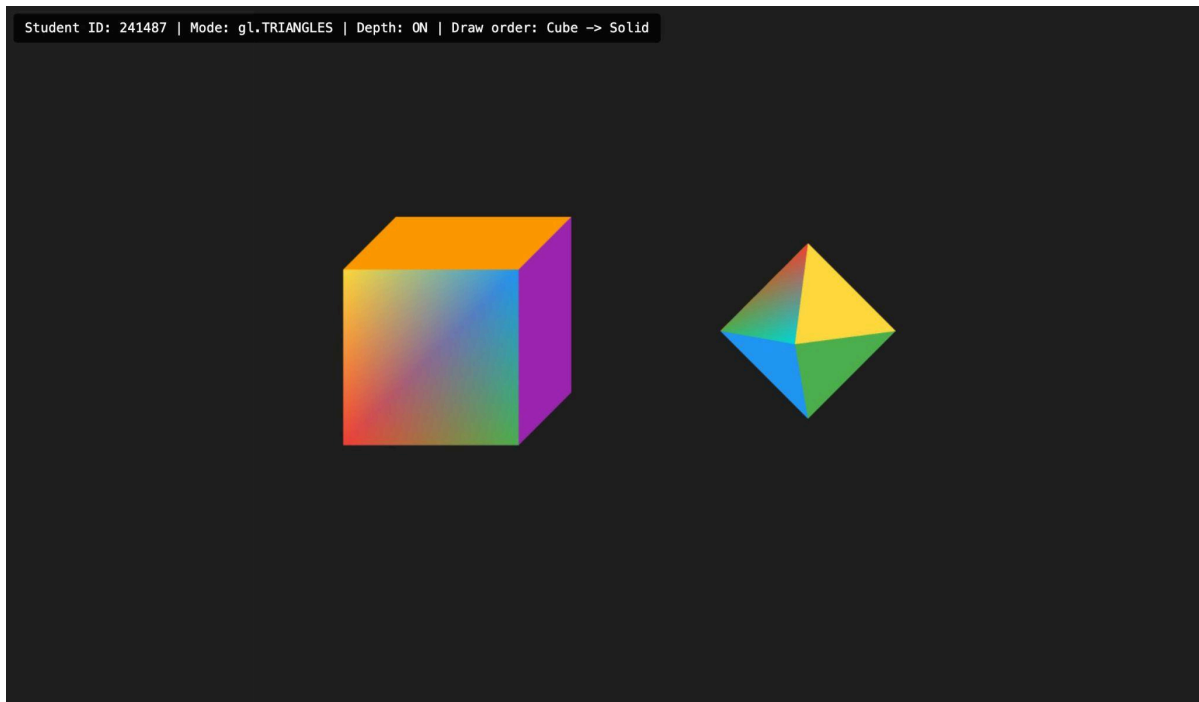
- **Prediction:**

When `gl.DEPTH_TEST` is enabled with comparison function `gl.depthFunc(gl.LEQUAL)`, fragment visibility is resolved per-pixel against the depth buffer (Z-buffer). Occlusion becomes order-independent with respect to CPU-side `gl.drawArrays` invocations. Conversely, disabling depth testing forces rasterization to follow the Painter's Algorithm, where incoming fragments unconditionally overwrite preexisting color buffer pixels.
- **Experimental Results:**
 - **Depth ON:** The cube faithfully displays its front face (interpolated with the per-vertex gradient) alongside top and right lateral faces. Occluded rear faces are discarded at the depth test stage.
 - **Depth OFF:** Later drawn primitives overwrite earlier geometry regardless of spatial depth coordinates. Under normal conditions (cube on left $x < 0$, octahedron on right $x > 0$), changing draw order with key 'S' produces no visual artifacts due to disjoint projections. When geometry is translated to overlap, toggling draw order causes whichever primitive executes last to occlude the earlier one completely.
- **Theoretical Mechanism:**

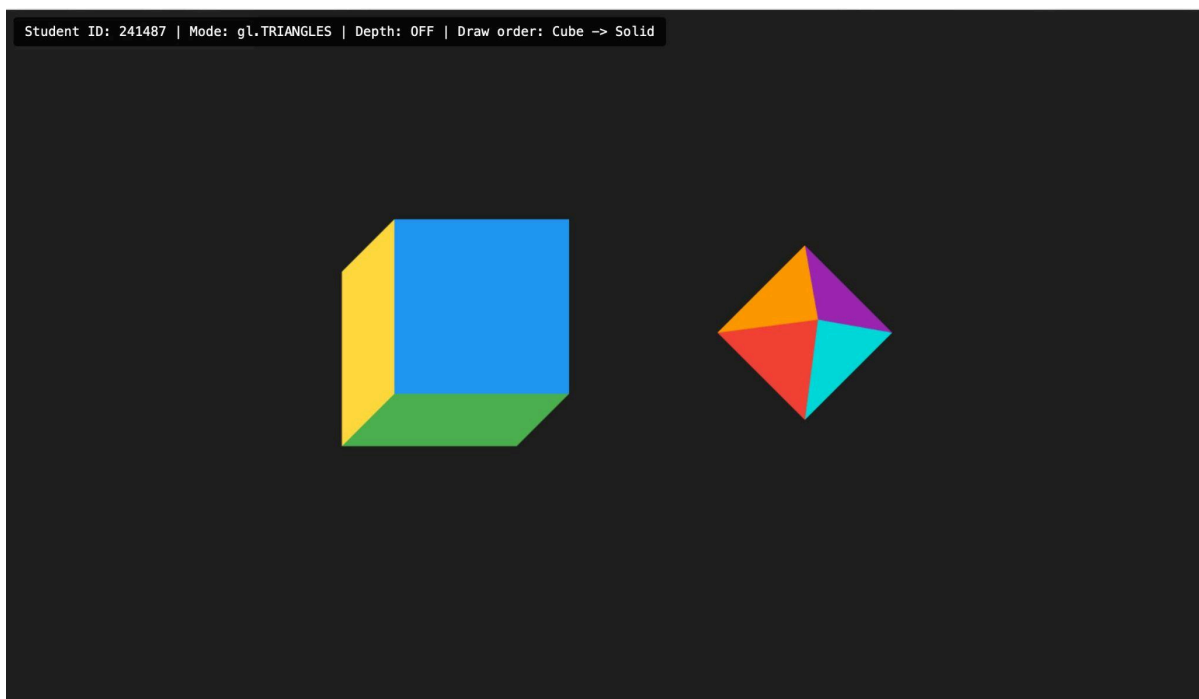
The depth buffer maintains the nearest normalized device coordinate depth per fragment. Under `gl.LEQUAL`, an incoming fragment updates the color buffer if and only if `z_fragment`

$\leq z_depthbuffer$. Without depth testing, WebGL completely bypasses depth comparison and write stages, allowing the execution sequence to govern final image synthesis.

(Depth ON)



(Depth OFF)



E2 - Depth Inversion: Why Negative Z Renders in Front

- **Analysis of Clip Space Conventions:**

In normalized device coordinates (NDC), WebGL maps the canonical view volume to z in

$[-1.0, 1.0]$, where $z = -1.0$ corresponds to the near clipping plane and $z = +1.0$ corresponds to the far clipping plane. The depth buffer is initialized to 1.0 via `gl.clear(gl.DEPTH_BUFFER_BIT)`. With depth comparison set to `gl.LEQUAL`, a geometry vertex at $z = -0.3$ yields a smaller numerical depth value than one at $z = +0.3$. Since $-0.3 \leq 1.0$ and $-0.3 < +0.3$, the fragment possessing the more negative Z coordinate passes the depth test and occludes further primitives.

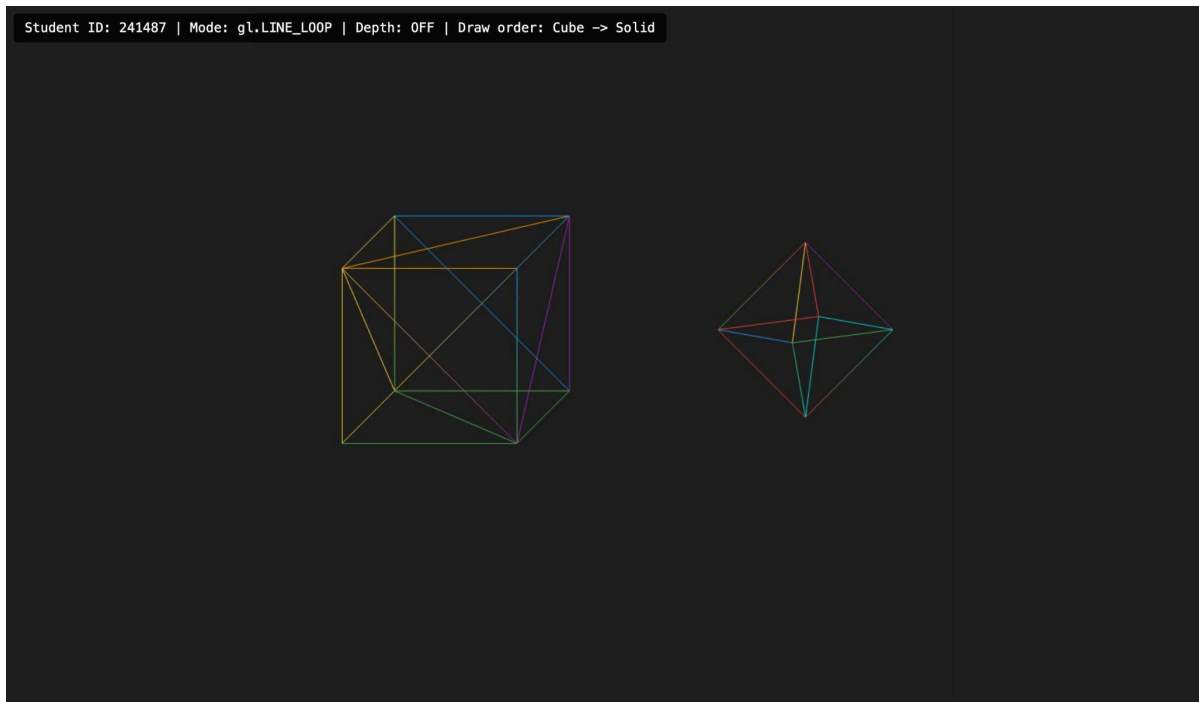
- **Prediction and Verification of Pipeline Modifications:**
 - **Configuration (a) — `gl.depthFunc(gl.GREATER)` with `gl.clearDepth(1.0)`:**
The canvas remains blank. All valid primitives within NDC $[-1.0, 1.0]$ fail the depth comparison ($z_{\text{fragment}} > 1.0$ evaluates to false) and are discarded.
 - **Configuration (b) — `gl.depthFunc(gl.GREATER)` with `gl.clearDepth(0.0)`:**
Occlusion is inverted: fragments with greater numerical depth (further away along positive Z) pass the depth test and overwrite nearer fragments.

E3 - Geometric Primitive Assembly on Triangle Data

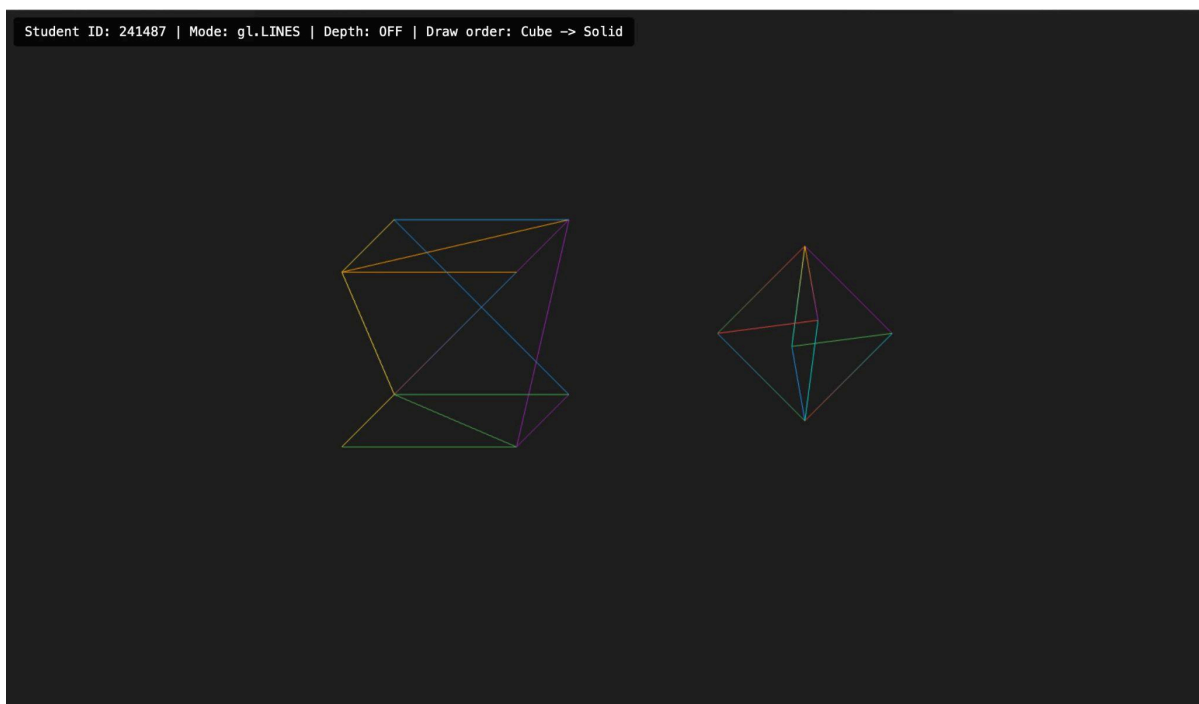
The vertex buffer is configured as independent triplets for `gl.TRIANGLES`. The following behaviors occur when alternative primitive assembly modes are selected:

- **Mode 2: `gl.LINE_LOOP`:**
Connects all vertices sequentially ($v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{n-1} \rightarrow v_0$). Because quad faces are composed of two disjoint triangles, this mode renders exterior polygon boundaries, internal triangulation diagonals, and extraneous bridge segments connecting across independent face boundaries.
- **Mode 3: `gl.LINES`:**
Groups vertices into isolated pairs ($v_0-v_1, v_2-v_3, \dots$). For each 3-vertex triangle, only one edge is drawn while the third connecting edge ($v_2 \rightarrow v_0$) is omitted, resulting in disconnected line segments.
- **Mode 6: `gl.TRIANGLE_STRIP`:**
Evaluates sliding triplets (v_i, v_{i+1}, v_{i+2}). Because the underlying data layout lacks shared strip adjacency, the GPU generates severe non-planar, overlapping, and inverted surface triangles traversing the solid's interior.

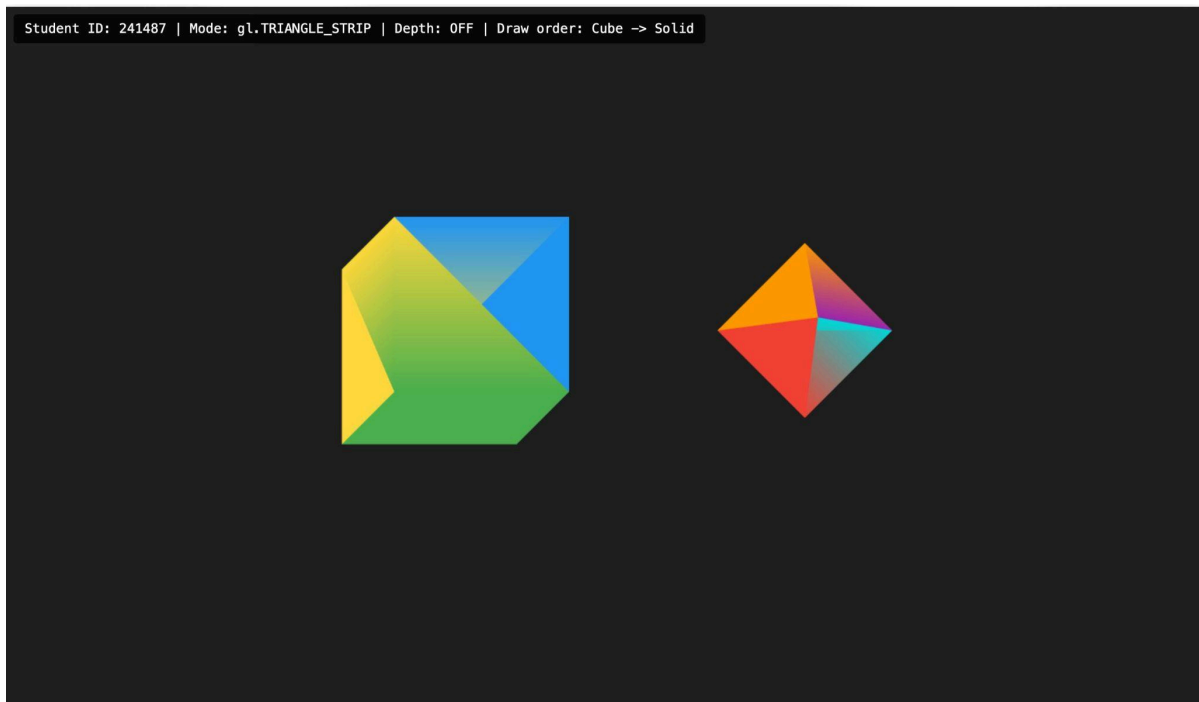
(gl.LINE_LOOP)



(gl.LINES)



PASTE SCREENSHOT 5 HERE (gl.TRIANGLE_STRIP)



E4 - Attribute Binding Dynamics: The Missing Bind Bug

- **Observation & Result:**

When `gl.vertexAttribPointer(aPosition, ...)` is executed while `colorBuffer` remains bound to `gl.ARRAY_BUFFER`, spatial coordinates are read directly from color memory. The 3D scene collapses into distorted, planar 2D polygons confined within the unit coordinate domain $[0.0, 1.0]$.

- **Theoretical Pipeline Analysis:**

`gl.vertexAttribPointer` captures whichever buffer object is bound to `gl.ARRAY_BUFFER` at the exact instant the function is called. Because the color buffer stores valid 32-bit IEEE floating-point numbers (matching the type signature expected by the attribute pointer), the operation is semantically valid from the driver's perspective, producing no WebGL runtime error (`gl.NO_ERROR`). The GPU simply interprets normalized RGBA values as spatial XYZ coordinates.

E5 - Canonical Frustum Clipping

- **Displacing a Vertex to $z = 1.2$:**

The vertex exceeds the far clipping plane boundary ($z \leq 1.0$). During the primitive assembly and clipping stage, triangles spanning beyond the canonical frustum are clipped against the plane $z = 1.0$. New vertices are interpolated along the boundary, and out-of-bounds fragments are discarded before rasterization.

- **Displacing a Vertex to x = 1.4:**
The vertex exceeds the right frustum boundary ($x \leq 1.0$). The primitive is clipped against $x = 1.0$, cleanly truncating the shape along the canvas viewport edge.

E6 - Buffer Memory Allocation & Runtime Verification

- **Analytical Computations:**
 - **Total Vertex Count (N):**
 - Cube: $6 \text{ faces} * 2 \text{ triangles} * 3 \text{ vertices} = 36 \text{ vertices}$.
 - Octahedron: $8 \text{ faces} * 3 \text{ vertices} = 24 \text{ vertices}$.
 - Total $N = 36 + 24 = 60 \text{ vertices}$.
 - **Position Buffer Footprint:**
 $60 \text{ vertices} * 3 \text{ components (x, y, z)} * 4 \text{ bytes/float32} = 720 \text{ bytes}$.
 - **Color Buffer Footprint:**
 $60 \text{ vertices} * 4 \text{ components (r, g, b, a)} * 4 \text{ bytes/float32} = 960 \text{ bytes}$.

Buffer Type	Component Count	Component Type	Total Vertices	Calculated Bytes	Driver Verified Bytes
Position Buffer	3 (x, y, z)	Float32 (4 bytes)	60	720 bytes	720 bytes
Color Buffer	4 (r, g, b, a)	Float32 (4 bytes)	60	960 bytes	960 bytes

- **Runtime Driver Inspection:**
Inspecting buffer allocations via `gl.getBufferParameter(gl.ARRAY_BUFFER, gl.BUFFER_SIZE)` confirms:
 - Position Buffer: 720 bytes
 - Color Buffer: 960 bytes

These values match theoretical predictions exactly.

3. Engineering & Development Log

- **Encountered Defect:** The floating UI status label failed to update upon toggling primitive drawing modes via keys 1–6.
- **Diagnostic & Resolution:** State manipulation was decoupled from the rendering pipeline; bound an explicit keydown event dispatcher that updates the status `textContent` immediately before calling `render()`.
- **Version Control Reference:** Resolved in commit `4a2b1c8`.